CYBR 437 Secure Coding

Fall 2026

Lab - 3

Release date: October 8, 2026

Due date: October 14, 2026, at 11:59 pm

Introduction

In this lab, you will practice using pwndbg, a Python-based extension of GDB, to analyze a binary and map out the stack and heap for a provided program.

Part 1 (5 points each task):

In this section, you'll be using *pwndgb* to perform some static analysis of the included binary (the C source files are not included and so commands like `list` will not be usable).

1. What is the name of the C file that contains main?
2. Check the security of the binary and display the output inside pwndbg.
3. Disassemble main and identify the user-defined functions (print statements and function calls in proper order).
4. Set a breakpoint at main and on each of the identified functions. Run the program.
5. What line are you at? What are the contents of the first string in main?
6. Display the current breakpoints, delete the one for main and set a new one at line 23, show the updated list of breakpoints.
7. Continue to the next breakpoint. What is the name and the value being passed into the function called?
8. How do you show the current function and the one that called it? Show output.
9. Disassemble this function you are in, how many print functions are in this function?
   Hint: There is more than one type of print statement.
10. Examining the disassembled code, grab the memory address of the constant from the function you're stopped at and display its location in the process image.
11. Continue running (`enter 15`). What function are you currently stopped at, what is the name and the value being passed into the function you are currently stopped at?
12. What is the value of the first string in this function you are stopped at, and what is the name and the type of the global variable in the function where you are stopped in?
    Hint: notice the legend.
13. Look for the `cmp` instruction in the assembly from the disassembled code. What value is the variable from question 14 being compared to?
14. What area of the process image is the variable *s* located, what storage class is it?
    Hint: notice the value of s as it enters the function that is called twice.
15. Display 64 addresses of the stack (NOTE: this is not *backtrace).*

Part 2 (25 points):

Some C code is provided on the next page as an image. You are not typing in any code. Using the addresses provided in the image below, draw the stack frames for both main and foo functions. Clearly label each variable and its memory location. (You may use Excel to organize your diagram and take a screenshot)

• NOTE: The top of the stack in your picture will be high memory.

```
ubuntu@ip-10-219-1-48:~/lectures/lecture2$ ./a.out
In main
x: 0x7fffdb7701e0
ptr: 0x7fffdb770218
val: 0x7fffdb7701e8
dptr: 0x7fffdb7701f0
array: 0x7fffdb770220
array[5]: 0x7fffdb770234
val2: 0x7fffdb7701f8
y:0x7fffdb7701e4
dptr2: 0x7fffdb770200
ptr2: 0x7fffdb770208
word: 0x7fffdb770210
word: 0x55e8969572a0

In function foo
&x = 0x7fffdb770180
 a = 0x7fffdb7701a0
&d = 0x7fffdb770188
&b = 0x7fffdb770198
&c = 0x7fffdb770190
&i = 0x7fffdb770184

b = 0x55e8969576f0, c = 0x55e8969576d0
```

Example of a Stack Drawing (not for any of the above addresses)

Variable	Memory	Location	
char *z	0xAAAABB110120	stack	High Memory
int *svar	0xAAAABB110118	stack	
28 bytes gap			
int q	0xAAAABB1100F8	stack	
gap to heap			
qq	0x55555555AA00	heap	Low Memory

```
void foo(void)
{
    int x = 256;
    int a[4] = {16, 32, 64, 128};
    double d = 512;
    int *c = malloc(16);
    int *b = (int *)calloc(32, sizeof(int));
    int i = 1024;

    printf("In function foo\n");
    printf("&x = %p\n a = %p\n&d = %p\n", &x, a, &d);
    printf("&b = %p\n&c = %p\n&i = %p\n\n", &b, &c, &i);
    printf("b = %p, c = %p\n", b, c);

    c = a;
    for (i = 0; i < 4; i++)
        a[i] = 100 + i;
    c[0] = 200;
    printf("a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
           a[0], a[1], a[2], a[3]);

    c[1] = 400;
    *(c + 2) = 402;
    3[c] = 404;
    printf("a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
           a[0], a[1], a[2], a[3]);

    c = c + 1;
    *c = 500;
    printf("a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
           a[0], a[1], a[2], a[3]);

    c = (int *) ((char *) c + 1);
    *c = 800;
    printf("a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
           a[0], a[1], a[2], a[3]);

    b = (int *) a + 1;
    c = (int *) ((char *) a + 1);
    printf("b = %p, c = %p\n", b, c);

}// end foo

int main(int argc, char **argv)
{
    int x = 12;
    double val = 4.9567;
    double *dptr = &val;
    int array[] = {10,20,30,40,50,60};
    double val2 = 3.14159;
    int y = 99;
    double * dptr2 = &val2;
    int *ptr2 = &y;
    char *word = (char*)calloc(10, sizeof(char));
    int *ptr = &x;

    printf("In main\n");

    printf("word: %p\n", word);
    printf("\n");

    foo();
    free(word);
    return 0;
}// end main
```

Submission Instructions: Please submit a single PDF file in Canvas that includes the correct pwndbg command, as applicable, screen capture(s) of the output, and your stack diagram.